CityCycle London Bike Rebalancing Intelligence Pipeline

Team 2

Good morning. We're from the Ops Analytics Team at CityCycle, a bike sharing company in London.

Our goal is to help solve some of the bike rebalancing issues that the operations team has been facing, by analysing usage behaviour and ride trends across the years 2020 to 2023.

We will use a modern ELT pipeline, to analyse the data, and then we will train a model to forecast how many bikes to add or remove from any station in order to meet operational efficiency and improve rider experience.

Here is an overview of our presentation.

*** NO NEED TO READ JUST PAUSE 2 SECONDS ***
We identify the problem, and provide a proposed solution with projected business impact

We will share the data strategy with the architecture, data flow diagram, analysis, and modeling

We will share some key insights and talk more about how we are measuring our data

Then we provide risks and mitigation

Finally we will provide a demo of our intelligence tool

01
Introduction

The Problem

London's CityCycle network struggles with chronic bike imbalance across 795 docking stations — stranded riders, overflow docks, and reactive crew operations.

01
Empty Stations

Riders arrive to find no bikes during peak commute hours. Lost rentals, frustrated customers, negative NPS.

02
Overflowing Docks

Returning riders cannot dock bikes — no empty slots. Bikes abandoned nearby, increasing theft and damage risk.

03
Reactive Rebalancing

Crews deploy reactively based on reports, not data. Inefficient routing wastes time, fuel, and crew hours.

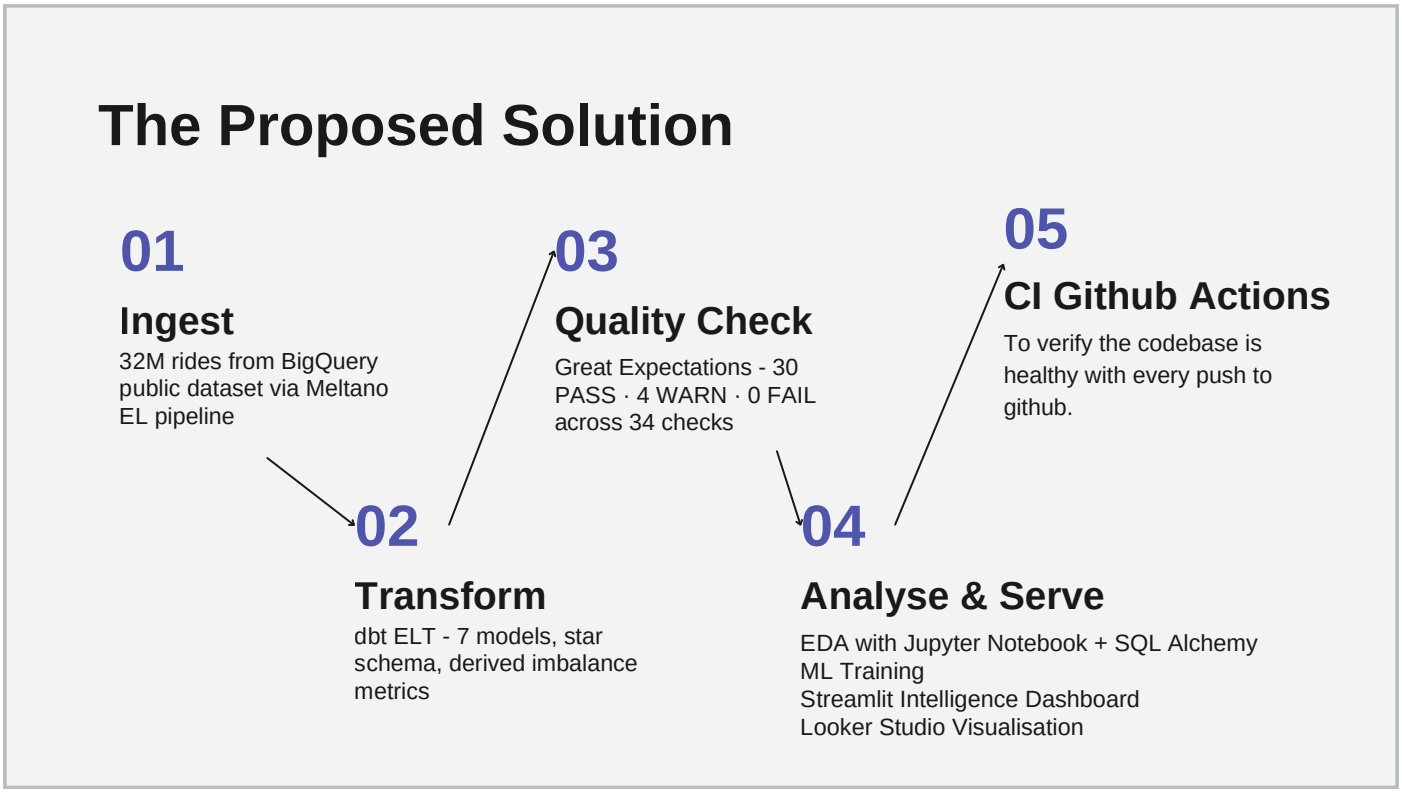
04
No Demand Forecasting

No predictive model for station-level demand. Operations cannot pre-position bikes before peak periods.

Here's the Problem -

CityCycle's London network operates 795 docking stations across the city, and every day some stations run completely empty during the morning commute while others overflow with bikes that can't be docked.

Riders arrive to find nothing. Returning riders can't dock. Crews are deployed reactively — based on complaints, not data — which wastes time, fuel, and crew hours.

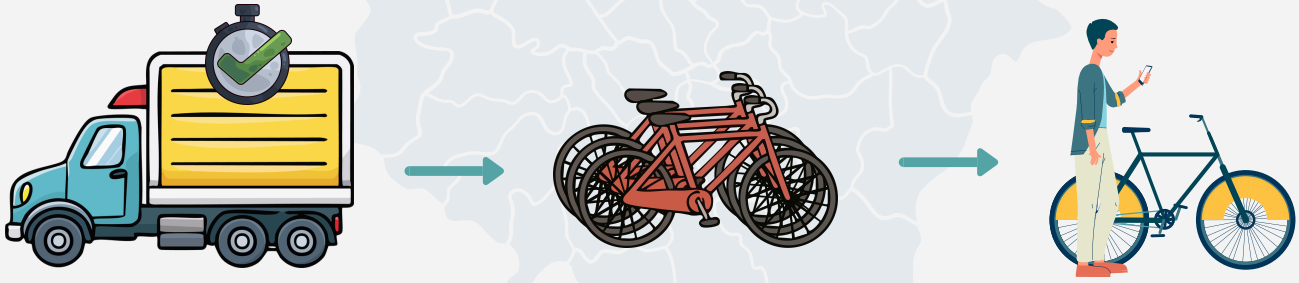


Our solution is an end-to-end ELT pipeline that ingests full 3-year ride history, transforms it into a star schema, runs automated quality checks, trains a demand forecasting model, and serves everything through an operational dashboard and executive reporting layer.

Business Impact

Data-driven rebalancing transforms CityCycle from **reactive to predictive operations**.

From the analysis we will be able to identify usage trends and station imbalances, leading to intelligent crew routing that reduces costs, prevents empty docks, and protects revenue - built on a **reusable ELT architecture that scales to any city**.

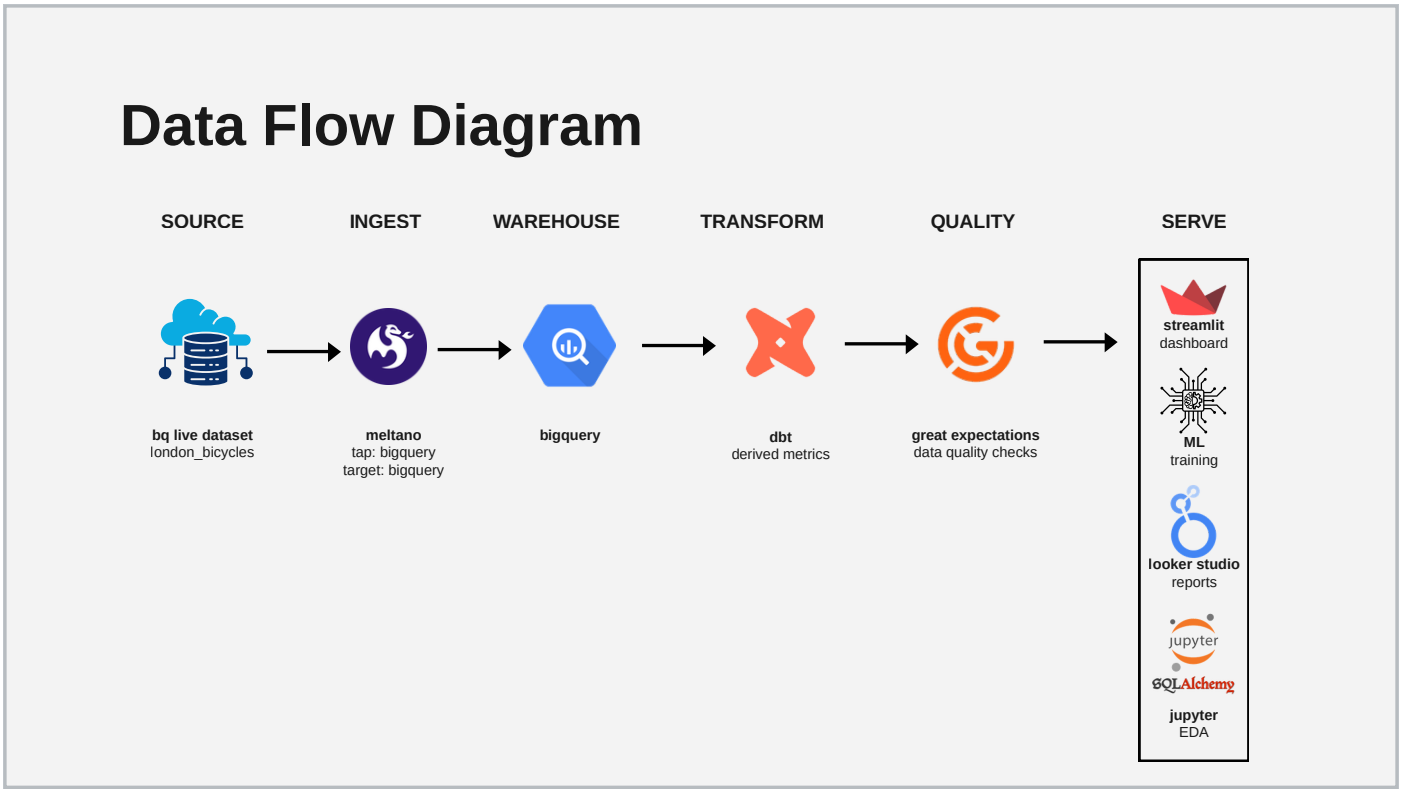


With data-driven rebalancing, CityCycle transforms from reactive to predictive operations.

By knowing which stations will drain before it happens, crews can be pre-positioned ahead of the AM peak or PM peak — rather than responding when bikes are already gone.

That protects revenue, reduces crew costs, and improves rider experience.

02
Data Strategy



Here's how the data flows. BigQuery public dataset is the source,

Meltano is the extraction and load layer using the Singer protocol
- tap-bigquery reads from london_bicycles
- target-bigquery writes to our project's raw dataset.

From there, dbt runs 7 transformation models across staging, intermediate, and marts layers, producing the star schema that everything downstream consumes.

Great Expectations gates quality at two points - post-ingest, and post-transform

And the output feeds to Streamlit, Looker Studio, and the Jupyter EDA notebook — all reading from the same fact_rides and dim_stations tables.

*** DON'T NEED TO READ BELOW FOR REF ONLY ***
Staging (2 models) — clean and type the raw data

stg_cycle_hire — renames columns, casts types, filters invalid rows from raw ride data
stg_cycle_stations — cleans and standardises station registry data

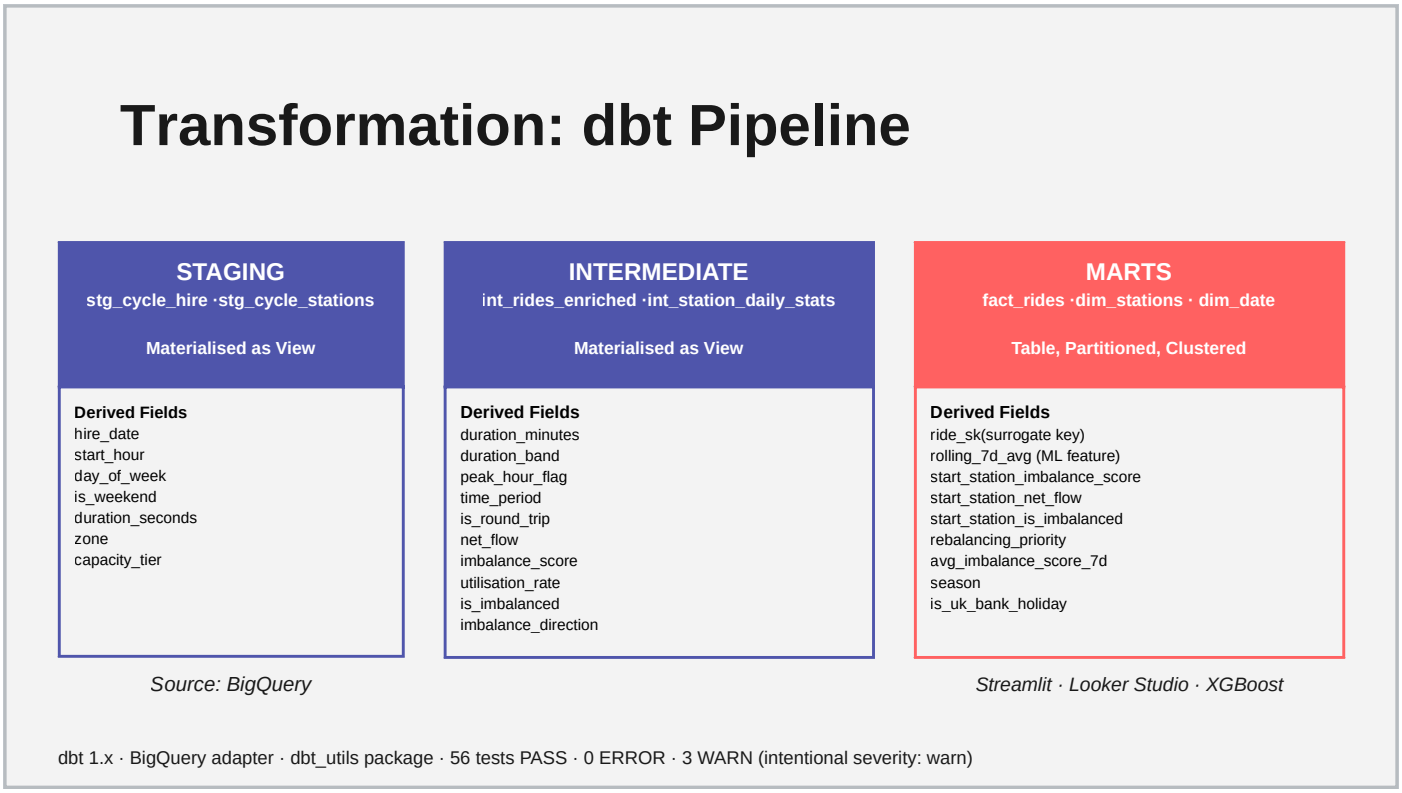
Intermediate (2 models) — apply business logic

int_rides_enriched — joins rides with station data, computes duration bands, peak hour flags, and

imbalance direction per ride
int_station_daily_stats — aggregates to station-day level, computing net flow and imbalance score per station per day

Marts (3 models) — materialise the final star schema

fact_rides — 32M row central fact table, partitioned by date, clustered by station, with all ML features and imbalance scores denormalised onto each ride row
dim_stations — one row per station with all-time imbalance averages and rebalancing priority tier
dim_date — date spine from 2015 to 2025 with season, weekday, and bank holiday flags for time-based joins

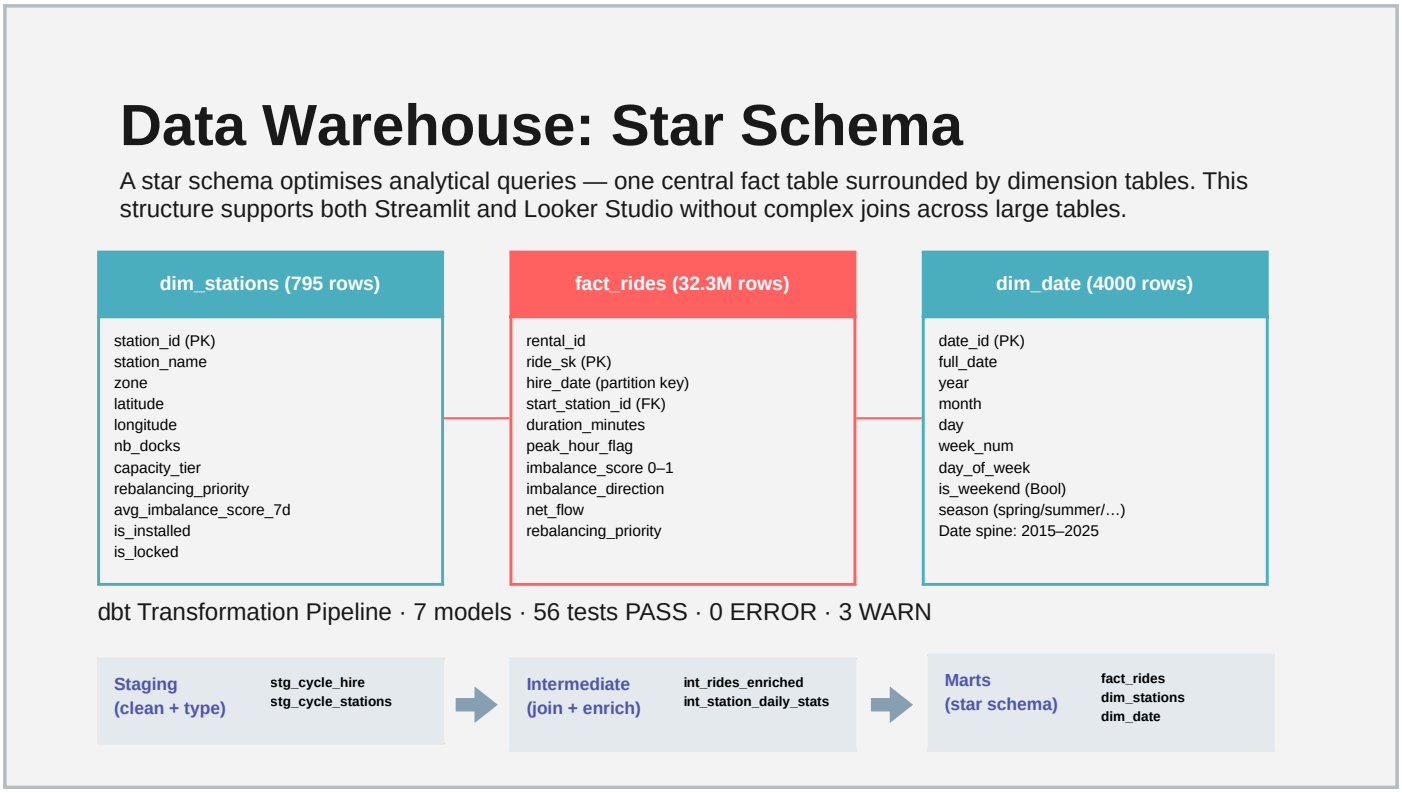


The transformation is where the analytical heavy lifting happens.

Staging cleans and types the raw data — renaming columns, casting types, filtering bad rows.

Intermediate applies business logic — computing peak hour flags, duration bands, and most importantly the imbalance score.

Marts materialise the final tables partitioned by date and clustered by station for fast queries.



Here is the final star schema of fact_rides, with 32.3M rows, dim_stations (795 rows), and dim_date (4000 rows)

Data Quality: Great Expectations

30	4	0	34
PASS	WARN	FAIL	TOTAL

Checkpoint 1 — Post-Ingest (citycycle_raw)	
PASS	rental_id — not null + unique
PASS	start_date — after 2010-01-01
PASS	start_station_id — not null
PASS	latitude/longitude — London range
PASS	total row count — at least 1M rows
WARN	duration < 60s — 27K docking errors (source data)
WARN	end_station_id NULL — 312K lost/unreturned bikes
WARN	end_date ≤ start_date — 10K timestamp errors (source)

Checkpoint 2 — Post-Transform (marts)	
PASS	ride_sk surrogate key — unique
PASS	duration_minutes — between 1 and 1440
PASS	start_hour — between 0 and 23
PASS	duration_band — valid values only
PASS	null rate on key columns — under 5%
PASS	end_datetime > start_datetime
PASS	imbalance_score — between 0 and 1
WARN	FK: start_station_id → dim_stations (hist. stations)

Two quality gates run automatically across the pipeline.

Checkpoint 1 validates the raw ingested data — nulls, uniqueness, coordinate ranges, and row counts.

Checkpoint 2 validates the dbt-transformed marts — surrogate keys, business logic ranges, and referential integrity.

All 4 warnings are known source data issues, deliberately downgraded from errors.

*** DON'T NEED TO READ FOR REF ONLY FAQ #11 ***
We downgraded to WARN because these are upstream source issues we cannot fix, the staging layer already filters them out before they reach the marts, and failing on known expected issues would permanently block the orchestration pipeline.

03
Key Insights

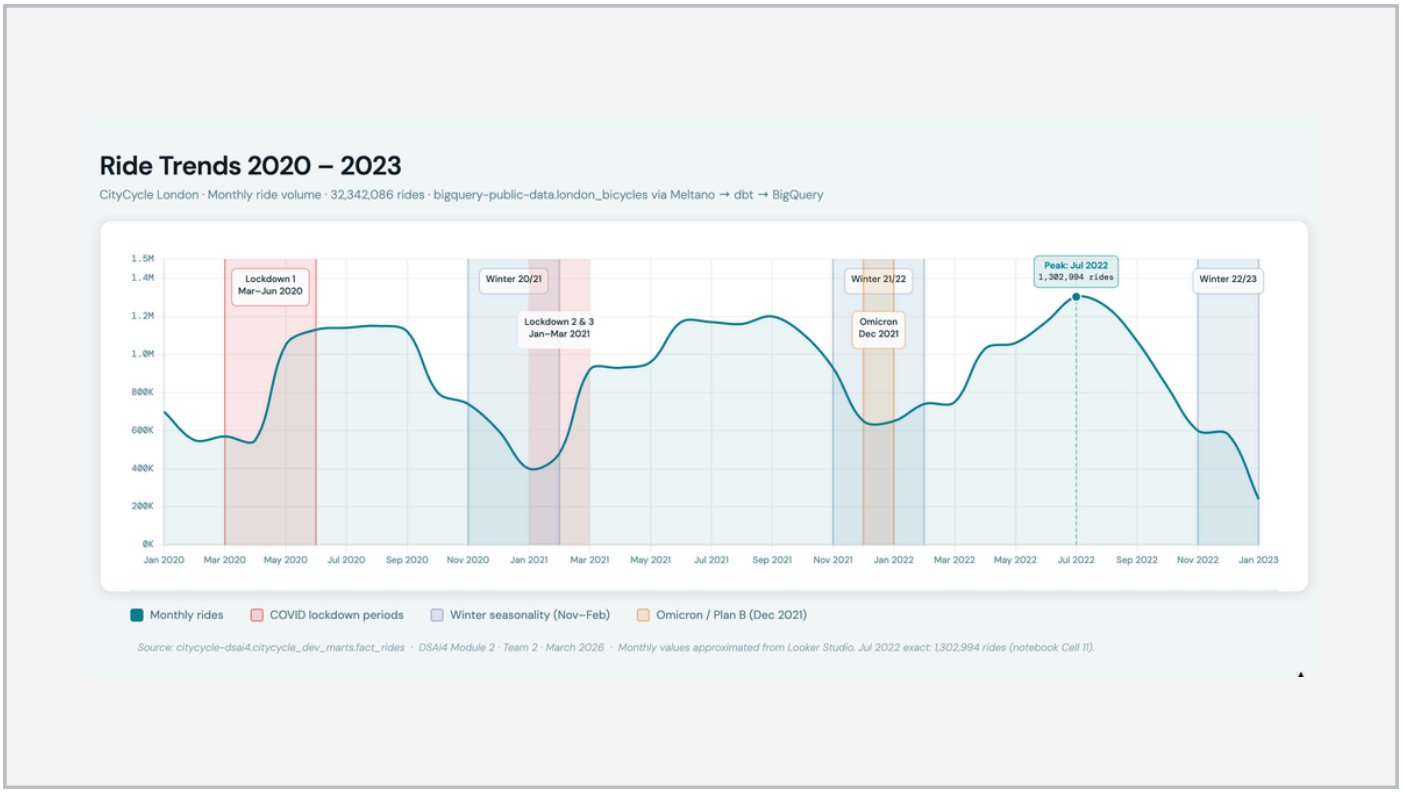
Key Findings

Metric	Value	Insight & Business Implication
Total rides (2020–2023)	32,342,086	3 years of real operational data — sufficient for trend detection and ML model training
Average ride duration	21.8 minutes	Mixed use confirmed — commuter and leisure trips coexist across the network
Rider segments	Leisure 53% · Casual 32% · Commuter 15%	K-Means k=3 — each segment drains stations in different directions at different times
Weekend rides	9.47M (29.3%)	Strong leisure demand — different rebalancing strategy required Sat/Sun vs weekdays
Avg network imbalance score	0.084	Network-wide baseline — scores above 0.18 trigger scheduled crew intervention
Critical stations	3 stations	Score ≥ 0.25: New North Road Hoxton · Ladbroke Grove Central · Cloudesley Road Angel
High priority stations	27 stations	Score ≥ 0.18 — require scheduled daily intervention by rebalancing crews
XGBoost RMSE	2.422 rides/hr	Best of 3 models tested — within operational planning threshold for crew dispatch
Top draining station	New North Road Hoxton	Score 0.324 · Net flow +7.0 bikes/day · Draining 90% of days — needs daily pre-AM crew
COVID-19 signal	Visible in 2020	Clear ridership collapse March–May 2020, full recovery to pre-pandemic levels by mid-2022

32 million rides across 3 years reveals a network with measurable imbalance patterns.

Average ride duration is about 21.8 minutes, mixed use.

Three rider segments — Leisure 53%, Casual 32%, Commuter 15% — each drain stations at different times and in different directions, enabling targeted crew scheduling. Only 30 of 795 stations are CRITICAL or HIGH priority. New North Road Hoxton is the worst offender — draining 7 bikes per day, 90% of days.



We also uncovered something interesting.

This ride trend chart showed that Ridership fell roughly 30% during the first UK lockdown in March-April 2020, then recovered sharply through summer.

Each subsequent summer peak was higher than the last — 1.1 million in summer 2020, 1.2 million in 2021, reaching a record 1.3 million rides in July 2022.

Full recovery and growth was also confirmed in the data.

The January 2021 trough is the deepest in the dataset — the UK's third lockdown coinciding with peak winter seasonality, two exogenous factors compounding simultaneously.

This is why the model explains 49% of variance — the rest is exogenous: policy, weather, seasonality. These are signals the pipeline captures but the ML model can't predict.

Understanding Imbalance

Imbalance Score = ABS(departures − arrivals) ÷ (departures + arrivals)

Range: 0.0 (perfectly balanced) → 1.0 (entirely one-directional) · Calculated daily per station ·CRITICAL threshold: 0.25

Filling	Balanced	Draining
net_flow < 0 More bikes arriving than leaving. Docks filling toward overflow. Crew Action: COLLECT BIKES — remove bikes to free up empty docking slots. *Hop Exchange, The Borough* *Score 0.15 · Net flow −38/day*	**net_flow ≈ 0** Departures ≈ arrivals. Station self-regulating. Crew Action: MONITOR ONLY — no immediate intervention required. *Score < 0.10* *Low priority station*	**net_flow > 0** More bikes leaving than arriving. Station heading toward empty. Crew Action: DELIVER BIKES — bring bikes from nearby filling stations. *New North Road Hoxton* *Score 0.324, net flow +7.0/day*

Why 0.25 = CRITICAL: A score of 0.25 means 25% more bikes moving one direction than the other. At this rate, a busy station risks running empty or reaching full capacity within a single day.
Thresholds were calibrated from the all-time score distribution: CRITICAL ≥ 0.25 · HIGH ≥ 0.18 · MEDIUM ≥ 0.10 · LOW < 0.10

The imbalance score is the single metric that drives everything — the dashboard, the priority tiers, and crew dispatch. It measures how one-directional a station's bike flow is on any given day, on a scale of 0 (perfectly balanced) to 1 (completely one-directional).

Every station falls into one of three states.
- Filling — more bikes arriving than leaving, crew collects.
- Draining — more bikes leaving than arriving, crew delivers.
- Balanced — no action needed.

We set CRITICAL at 0.25 because at that level, a busy station can run completely empty or overflow within a single day.

The four tiers — CRITICAL, HIGH, MEDIUM, LOW — were calibrated from 853,000 real station-days of data, giving operations teams a daily ranked list of exactly where to go and what to do.

*** DON'T READ FOR REF ONLY ***

The 853,000 station-days come from int_station_daily_stats — our intermediate dbt model that aggregates the 32 million individual rides up to station-day level. Each row represents one station on one day, with its computed imbalance score. That's the distribution we used to calibrate the priority thresholds.

04
Risk Mitigation

Decisions & Risk Management

KEY RISKS & MITIGATIONS

- **BQ free tier exceeded (1TB/month)**
Dry-run cost checks before every query · LIMIT guards · date partitioning · budget alert at 80%

- **Source schema drift (TfL updates)**
dbt schema tests catch column changes on next run · GE post-ingest checkpoint adds second layer

- **Source data quality (public dataset)**
312K null end_station_id · 27K short durations · 10K timestamp errors — all flagged as GE WARN not FAIL

FAILURES ENCOUNTERED & FIXED

- **BOOL/INT64 type mismatch in BigQuery**
COUNTIF(col) replaced with SUM(CAST(col AS INT64)) across all queries, notebooks and dashboard pages
- **dim_stations all showing LOW priority**
7-day WHERE filter in latest_stats CTE returned zero matches · Fixed with all-time averages · Thresholds recalibrated
- **XGBoost feature order mismatch**
season column in wrong position between training and dashboard · Explicit column reorder enforced in CI and dashboard
- **APPROX_QUANTILES breaks SQLAlchemy**
Returns ARRAY type which SQLAlchemy cannot deserialise · Replaced with AVG throughout

MOCK-FIRST DEVELOPMENT

- **Why mock-first?**
london_bicycles has 83M rows — one unguarded SELECT * consumes the entire 1TB monthly free tier instantly
- **10K synthetic rides (numpy, seed=42)**
Schema matches real BQ tables exactly - same columns, types and constraints.
- **CI/CD runs zero live BQ calls**
All 5 GitHub Actions jobs use mock CSV only · dbt compile validates SQL at zero cost · No credentials in CI
- **Dev toggle in dashboard**
Streamlit use_mock switch — one click to test against mock before hitting live BQ
- **Outcome: $0 development cost**
Entire pipeline built and validated without triggering BQ billing · Live run completed safely in one production pass

KEY RISKS & MITIGATIONS

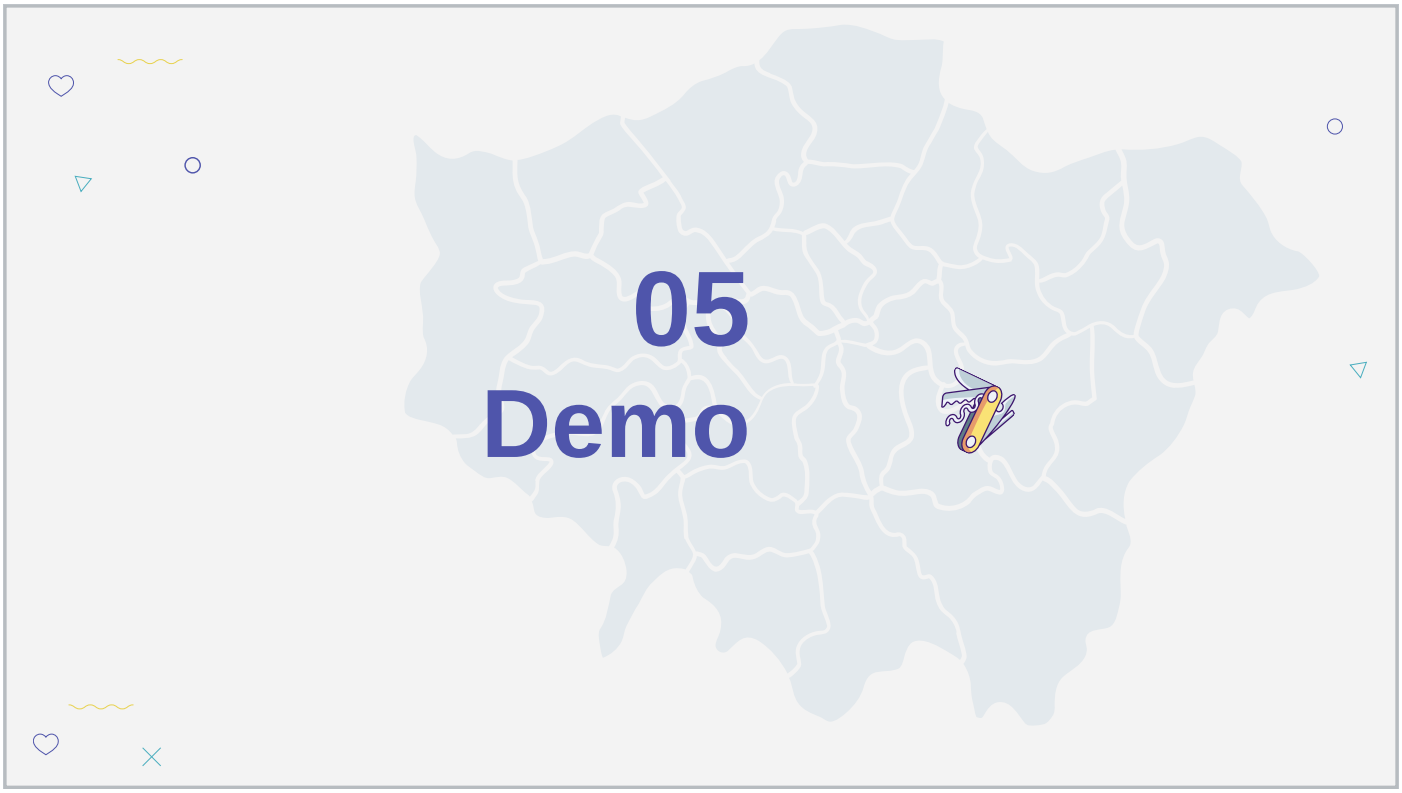
BigQuery's 1TB free tier was the primary cost risk — mitigated with dry-run guards, partitioning, and mock-first development. Source data quality issues (312K null end stations, 27K docking errors) were anticipated and documented as intentional GE warnings. Credentials are fully protected via .gitignore.

FAILURES ENCOUNTERED & FIXED

Three silent failures hit during development: BOOL/INT64 type mismatches returning zero, every station showing LOW priority due to a CTE filter bug, and XGBoost producing wrong forecasts from a feature column mismatch. All diagnosed, fixed, and hardened into CI validation.

MOCK-FIRST DEVELOPMENT

The entire pipeline was built against 10K synthetic rides before touching live BigQuery — zero development cost, zero credential exposure in CI. When switched to live data, 32 million rows ran cleanly first time. Mock-first made safe iteration possible on a constrained free tier.

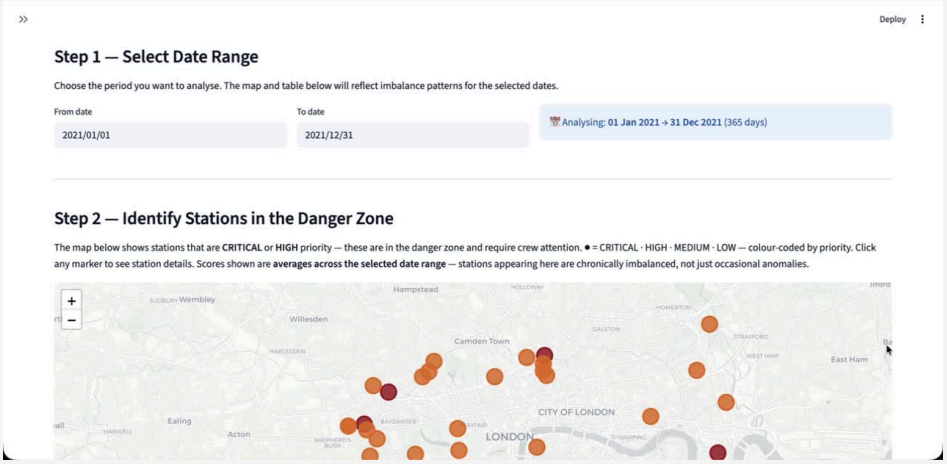


Let me now show you the scenario planner — this is where everything comes together for an operations team.

Scenario Planner

COMMAND:
streamlit run
dashboard/app.py

Audience: Operations teams need interactivity and real-time filters.
Purpose: Interactive exploration and crew dispatch planning



Step 1 — Select date range
Choose the analysis period. Map and table filter to that window — summer and winter behave very differently.

Step 2 — Identify danger zone stations
795 stations colour-coded by priority. Red = CRITICAL, orange = HIGH. Inner London concentration is immediately visible.

Step 3 — Filter to CRITICAL and HIGH
Narrow to the 30 stations needing daily attention. Table shows imbalance score, direction, and bikes needed.

Step 4 — Understand draining vs filling
Draining = deliver bikes. Filling = collect bikes. Quantity comes directly from net flow — no manual calculation.

Step 5 — Review the dispatch list
Filtered table becomes the crew morning briefing. In production, this runs automatically every day.

Step 6 — Forecast hourly demand
XGBoost predicts departures per station per hour. RMSE 2.422 — precise enough for crew scheduling decisions.

Future Improvements & Next Steps

01

Enrich the feature set

Real-time weather and live dock data improves forecast accuracy.

02

Segment-aware rebalancing schedules

Pre-position bikes by rider type, not just imbalance score.

03

Predictive dock capacity alert

Flag stations running short before the morning peak hits.

04

Expand to other cities

Same pipeline, different dataset - Dublin, New York, Paris ready.

05

Automate the full production pipeline

Activate Dagster schedule - nightly runs, no manual intervention needed.

*** NO NEED TO READ JUST READ THE TITLES FROM THE SLIDE ***

1. Enrich the feature set to improve forecast accuracy
The XGBoost model currently explains 49% of variance. The remaining 51% is largely driven by factors not in our data — weather, live dock occupancy, and external events. Integrating TfL's live BikePoint API for real-time bike counts, weather data from Met Office or OpenWeatherMap, and UK event calendars would give the model the exogenous signals it currently lacks. A well-enriched feature set could realistically push R^2 above 0.70.

2. Build segment-aware rebalancing schedules
The K-Means analysis already tells us that commuter stations drain on weekday mornings and evenings, while leisure stations surge on weekends. The next step is to operationalise this — automatically tagging stations by their dominant rider segment and generating differentiated crew schedules: pre-AM stocking for commuter-dominant stations on weekdays, mid-morning restocking for leisure stations on Saturdays and Sundays. This moves the system from reactive prioritisation to proactive, segment-timed dispatch.

3. Add a predictive dock capacity alert
Currently the dashboard shows historical imbalance. A natural extension is a forward-looking alert — if today's forecast predicts 20 departures from a station with 12 bikes docked, surface that shortfall automatically as a same-day crew alert rather than waiting for the imbalance score to accumulate over multiple days.

4. Expand to other cities
The pipeline architecture is city-agnostic. The dbt models, GE checks, and Streamlit dashboard are parameterised by dataset — pointing the Meltano tap at a different city's bike-share public dataset would replicate the entire intelligence pipeline with minimal rework. Dublin, New York Citi Bike, and Paris Vélib all publish comparable public datasets.

5. Automate the full production pipeline
The Dagster orchestration currently runs as a POC against mock data. The production swap — replacing mock_bq_load_asset with meltano_ingest_asset and activating the citycycle_daily_02utc schedule — would make the pipeline fully automated, running nightly and keeping the dashboard current without manual intervention.

Demo Links

Looker Studio:
https://lookerstudio.google.com/s/hNwE4QyCuCg_

GitHub Repo:
https://github.com/berniepng/dsai4-m2-t2-citycycle

Streamlit:
streamlit run dashboard/app.py, OR open ~/docs/dashboard/05_scenario_static.html

Jupyter Notebook:
open ~/analysis/notebooks/03_bq_eda_live_data_executed.ipynb in IDE

We only had time to show you the Streamlit dashboard.

We also looker studio for executive reporting and the Notebook to show our analysis if you want to see more.

Thank you. Any questions?

06
Questions?